Advanced Software Engineering – Assignment Notes

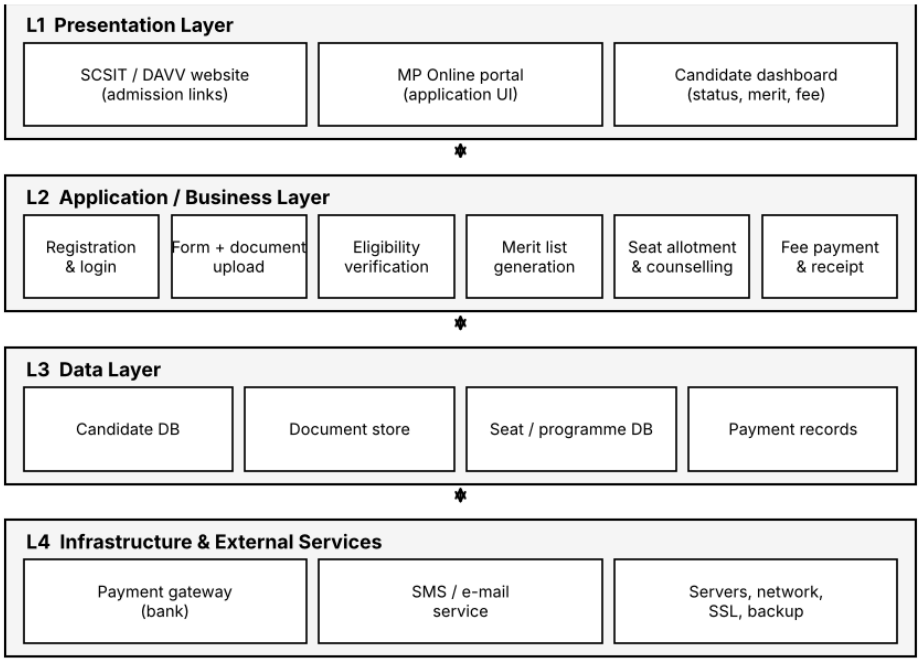
Rohit Bhure | Roll No. 2610611 | M.Tech (IASE), Semester 1 | SCS&IT, DAVV Indore | 2026–2027

Section A: Handwritten

Q1. Reusable libraries used by the software industry

#	Library	Language	Reused for
1	Apache Commons (Lang, IO, Collections)	Java	String, file and collection utilities
2	Google Guava	Java	Collections, caching, validation
3	Spring Framework	Java	Dependency injection, web, data access
4	JUnit	Java	Unit testing
5	React	JavaScript	Reusable UI components
6	NumPy / pandas	Python	Numeric arrays, data analysis
7	Boost / C++ STL	C++	Containers, algorithms, smart pointers
8	.NET Base Class Library	C#	Collections, I/O, networking

Q2. Layered architecture of the SCSIT admission process



Layered architecture: SCSIT website → MP Online portal → business processes → data → infrastructure

Layer	Systems	Subsystems / lower-level configuration
L1 Presentation	SCSIT website, MP Online portal	Admission links, application form UI, candidate dashboard; browser, HTTPS
L2 Business	Admission processing system	Registration/login, form + document upload, eligibility check, merit list, counselling and seat allotment, fee payment
L3 Data	Database servers	Candidate DB, document store, seat/programme DB, payment records; backup
L4 Infrastructure	External + hosting	Payment gateway (bank), SMS/e-mail, web/app servers, network, SSL, firewall

Layers are drawn from the typical online-admission flow; check the exact links on the live SCSIT / MP Online pages before submitting.

Q3. Central CASE repository and CASE tools

Importance of a central repository:

- Single source of truth: all models, requirements, data dictionary and code artefacts stored in one place.
- Consistency and completeness checking across diagrams (DFD, ERD, UML).
- Traceability and impact analysis: requirement → design → code → test.
- Reuse of stored components and models; team sharing with version control.
- Automatic documentation and code generation from the stored models.

Category	Five tools
Upper CASE (analysis, design)	StarUML, Enterprise Architect, Visual Paradigm, IBM DOORS, draw.io

Lower CASE (coding, testing, maintenance)	Eclipse, IntelliJ IDEA, JUnit, Selenium, SonarQube
Integrated CASE (whole life cycle)	IBM Rational Rose / Rational Software Architect, Visual Studio with Azure DevOps, Oracle Designer, SAP PowerDesigner, Enterprise Architect

Q4. Software maintenance categories with examples

Category	Meaning	Example
Corrective	Fix defects found in use	Fixing a bug where the fee receipt shows the wrong amount
Adaptive	Adapt to a changed environment	Changing the portal to a new payment gateway API or a new OS version
Perfective	Add or improve features	Adding a search filter and faster merit list report
Preventive	Prevent future problems	Refactoring code and adding unit tests to the admission module

Q5. Five project repositories

#	Repository	Domain	Why study it
1	github.com/apache/commons-lang	Java utility library	Well-packaged reusable components
2	github.com/google/guava	Java core libraries	Generic, reusable collection design
3	github.com/spring-projects/spring-framework	Enterprise framework	Layered architecture, dependency injection
4	github.com/junit-team/junit5	Testing framework	Extension/plug-in based reuse
5	github.com/apache/tomcat	Web / servlet container	Large layered, component-based system

Section B: Code

Q1. Generalized programs: add numbers, join strings, combine paragraphs, add objects, combine images

Idea: one generic operation *combine* – Java generics, C++ templates / operator overloading, R S3 generic. Images are combined side by side.

Java

```
import java.awt.image.BufferedImage;
import java.util.*;

public class Generic {
    // generic combine: works for numbers, strings, paragraphs, objects
    interface Combinable<T> { T combine(T other); }

    static class Money implements Combinable<Money> {          // "add two objects"
        int rs; Money(int rs) { this.rs = rs; }
        public Money combine(Money o) { return new Money(rs + o.rs); }
        public String toString() { return "Money(" + rs + ")"; }
    }
    static <T extends Combinable<T>> T add(T a, T b) { return a.combine(b); }
    static <T extends Number> double add(T a, T b)   { return a.doubleValue() + b.doubleValue(); }
    static String join(String a, String b)       { return a + b; }
    static String paragraphs(String p1, String p2) { return p1 + "\n\n" + p2; }

    static BufferedImage combine(BufferedImage a, BufferedImage b) {   // side by side
        BufferedImage r = new BufferedImage(a.getWidth() + b.getWidth(),
                Math.max(a.getHeight(), b.getHeight()), BufferedImage.TYPE_INT_RGB);
        r.getGraphics().drawImage(a, 0, 0, null);
        r.getGraphics().drawImage(b, a.getWidth(), 0, null);
        return r;
    }
    public static void main(String[] args) {
        System.out.println(add(5, 7) + " | " + add(2.5, 4.5));
        System.out.println(join("Soft", "ware"));
        System.out.println(paragraphs("Para one.", "Para two."));
        System.out.println(add(new Money(100), new Money(50)));
        BufferedImage i = combine(new BufferedImage(40, 30, 1), new BufferedImage(60, 30, 1));
        System.out.println("Image: " + i.getWidth() + "x" + i.getHeight());
    }
}
```

```
Output:
12.0 | 7.0
Software
Para one.

Para two.
Money(150)
Image: 100x30
```

C++

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
```

```
template <typename T> T add(T a, T b) { return a + b; }   // numbers, strings, paragraphs, objects

struct Money {                                            // add two objects
    int rs; Money(int r) : rs(r) {}
    Money operator+(const Money& o) const { return Money(rs + o.rs); }
};
string paragraphs(const string& a, const string& b) { return a + "\n\n" + b; }

struct Image { int w, h; vector<int> px; };               // simple grey-scale image
Image combine(const Image& a, const Image& b) {           // side by side
    Image r{a.w + b.w, max(a.h, b.h), {}};
    r.px.assign(r.w * r.h, 0);
    for (int y = 0; y < a.h; y++) for (int x = 0; x < a.w; x++) r.px[y*r.w + x] = a.px[y*a.w + x];
    for (int y = 0; y < b.h; y++) for (int x = 0; x < b.w; x++) r.px[y*r.w + a.w + x] = b.px[y*b.w + x];
    return r;
}
int main() {
    cout << add(5, 7) << " " << add(2.5, 4.5) << endl;
    cout << add<string>("Soft", "ware") << endl;
    cout << paragraphs("Para one.", "Para two.") << endl;
    cout << add(Money(100), Money(50)).rs << endl;
    Image a{2, 2, {1,1,1,1}}, b{3, 2, {2,2,2,2,2,2}};
    Image c = combine(a, b);
    cout << "Image: " << c.w << "x" << c.h << endl;
}
```

```
Output:
12 7
Software
Para one.

Para two.
150
Image: 5x2
```

R

```
# S3 generic: one function name, behaviour chosen by object type
combine <- function(a, b, ...) UseMethod("combine")
combine.numeric   <- function(a, b, ...) a + b
combine.character <- function(a, b, ...) paste0(a, b)
combine.money     <- function(a, b, ...) structure(list(rs = a$rs + b$rs), class = "money")
combine.matrix    <- function(a, b, ...) cbind(a, b)        # image = matrix of pixels

paragraphs <- function(p1, p2) paste(p1, p2, sep = "\n\n")

print(combine(5, 7)); print(combine(2.5, 4.5))
print(combine("Soft", "ware"))
cat(paragraphs("Para one.", "Para two."), "\n")
m <- combine(structure(list(rs = 100), class = "money"), structure(list(rs = 50), class = "money"))
print(m$rs)
img <- combine(matrix(1, 2, 2), matrix(2, 2, 3))           # 2 x 5 image
print(dim(img))
```

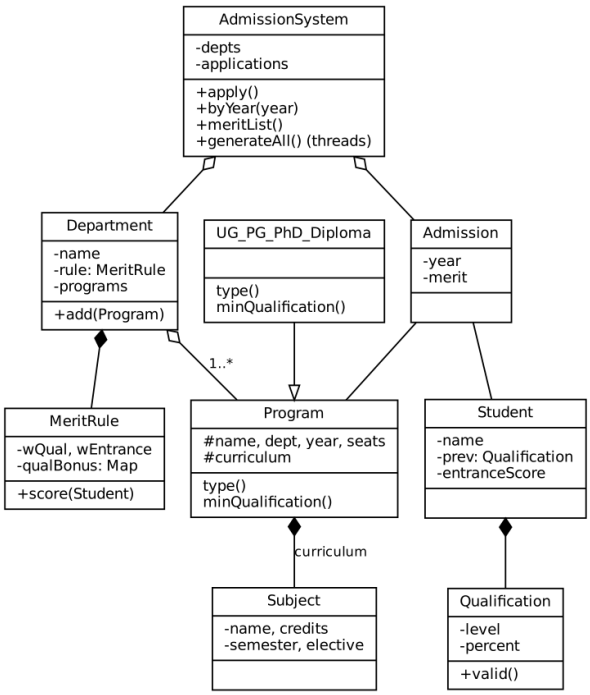
```
Output:
[1] 12
[1] 7
[1] "Software"
Para one.

Para two.
[1] 150
[1] 2 5
```

Java and C++ were compiled and run. R was not run here (R not installed); output shown is the expected result.

Q2. Student Admission and Program Registration System (Java)

Requirement	Where implemented
Course / program types	Abstract `Program`; subclasses UG, PG, PhD, Diploma (inheritance); each has department + year
Year-wise admission	`Admission.year`, `byYear(year)`; GUI year selector
Merit-based admission	`MeritRule` weights, qualification bonus (preference), `meritList()`
Previous qualification	`Qualification` validated in `apply()`, used in score
Curriculum	`Program.curriculum` list of `Subject` (credits, semester, elective)
Multiple departments	`Department` owns programs, seats, merit rule; shared Student/Admission classes
Threads, GUI	`generateAll()` runs one thread per program; Swing window runs it on a background thread



Class diagram of the admission system

```
import java.util.concurrent.*;
import java.util.*;
import java.util.stream.*;
import javax.swing.*;
import java.awt.BorderLayout;

// ---------- Inheritance: program types ----------
abstract class Program {
    String name, dept; int year, seats; List<Subject> curriculum = new ArrayList<>();
    Program(String name, String dept, int year, int seats) { this.name = name; this.dept = dept; this.year = year;
this.seats = seats; }
    abstract String type();
    abstract String minQualification();
}
class UG extends Program { UG(String n,String d,int y,int s){super(n,d,y,s);} String type(){return "UG";} String m
inQualification(){return "12th";} }
class PG extends Program { PG(String n,String d,int y,int s){super(n,d,y,s);} String type(){return "PG";} String m
inQualification(){return "Graduation";} }
class PhD extends Program { PhD(String n,String d,int y,int s){super(n,d,y,s);} String type(){return "PhD";} Strin
g minQualification(){return "Post-Graduation";} }
class Diploma extends Program { Diploma(String n,String d,int y,int s){super(n,d,y,s);} String type(){return "Dipl
oma";} String minQualification(){return "10th";} }

// ---------- Curriculum ----------
class Subject {
    String name; int credits, semester; boolean elective;
    Subject(String n, int c, int s, boolean e) { name = n; credits = c; semester = s; elective = e; }
}

// ---------- Qualification & Student ----------
class Qualification {
    String level; double percent;                 // level: 12th, Graduation, ...
    Qualification(String l, double p) { level = l; percent = p; }
    boolean valid() { return percent >= 0 && percent <= 100; }
}
class Student {
    String name; Qualification prev; double entranceScore;
    Student(String n, Qualification q, double e) { name = n; prev = q; entranceScore = e; }
}

// ---------- Admission (year-wise) ----------
class Admission {
    Student student; Program program; int year; double merit;
    Admission(Student s, Program p, int y) { student = s; program = p; year = y; }
}

// ---------- Merit rules (configurable, per department) ----------
class MeritRule {
    double wQual = 0.6, wEntrance = 0.4;                        // configurable weightage
    Map<String, Double> qualBonus = new HashMap<>();            // qualification preference rules
    double score(Student s) {
        double bonus = qualBonus.getOrDefault(s.prev.level, 0.0);
        return wQual * s.prev.percent + wEntrance * s.entranceScore + bonus;
    }
}

// ---------- Department: manages its own programs, seats, rules ----------
```

```
class Department {
    String name; MeritRule rule = new MeritRule();
    List<Program> programs = new ArrayList<>();
    Department(String n) { name = n; }
    Program add(Program p) { programs.add(p); return p; }
}

// ---------- Shared infrastructure ----------
class AdmissionSystem {
    List<Department> depts = new ArrayList<>();
    List<Admission> applications = new ArrayList<>();

    boolean apply(Student s, Program p, int year) {
        if (!s.prev.valid() || !s.prev.level.equals(p.minQualification())) return false;   // validate
        applications.add(new Admission(s, p, year)); return true;
    }
    Department deptOf(Program p) { return depts.stream().filter(d -> d.programs.contains(p)).findFirst().get(); }

    List<Admission> byYear(int year) {
        return applications.stream().filter(a -> a.year == year).collect(Collectors.toList());
    }
    // merit list for one program & year: top 'seats' applicants
    List<Admission> meritList(Program p, int year) {
        MeritRule r = deptOf(p).rule;
        List<Admission> l = byYear(year).stream().filter(a -> a.program == p).collect(Collectors.toList());
        l.forEach(a -> a.merit = r.score(a.student));
        l.sort((a, b) -> Double.compare(b.merit, a.merit));
        return l.subList(0, Math.min(p.seats, l.size()));
    }
    // multithreading: one thread per program
    Map<Program, List<Admission>> generateAll(int year) throws InterruptedException {
        Map<Program, List<Admission>> out = new ConcurrentHashMap<>();
        List<Thread> ts = new ArrayList<>();
        for (Department d : depts) for (Program p : d.programs) {
            Thread t = new Thread(() -> out.put(p, meritList(p, year)), "merit-" + p.name);
            ts.add(t); t.start();
        }
        for (Thread t : ts) t.join();
        return out;
    }

    // ---------- demo data ----------
    static AdmissionSystem sample() {
        AdmissionSystem sys = new AdmissionSystem();
        Department cs = new Department("SCSIT"), mg = new Department("Management");
        cs.rule.qualBonus.put("Graduation", 2.0);
        Program mtech = cs.add(new PG("M.Tech (IASE)", "SCSIT", 2026, 2));
        mtech.curriculum.add(new Subject("Advanced Software Engineering", 4, 1, false));
        mtech.curriculum.add(new Subject("Information System Design", 4, 1, false));
        mtech.curriculum.add(new Subject("Machine Learning", 3, 2, true));
        Program btech = cs.add(new UG("B.Tech (CS)", "SCSIT", 2026, 1));
        Program mba = mg.add(new PG("MBA", "Management", 2026, 1));
        sys.depts.add(cs); sys.depts.add(mg);
        Qualification g1 = new Qualification("Graduation", 78), g2 = new Qualification("Graduation", 85), g3 = new
Qualification("Graduation", 70);
        sys.apply(new Student("Rohit", g1, 80), mtech, 2026);
        sys.apply(new Student("Anjali", g2, 60), mtech, 2026);
        sys.apply(new Student("Mahesh", g3, 90), mtech, 2026);
        sys.apply(new Student("Neha", new Qualification("12th", 88), 70), btech, 2026);
        sys.apply(new Student("Karan", g2, 75), mba, 2026);
        sys.apply(new Student("Old", g1, 50), mtech, 2025);
        return sys;
    }

    // ---------- GUI (Swing) ----------
    static void gui() {
        AdmissionSystem sys = sample();
        JFrame f = new JFrame("Student Admission & Program Registration");
        JComboBox<Integer> yr = new JComboBox<>(new Integer[]{2025, 2026});
        yr.setSelectedItem(2026);
        JTextArea out = new JTextArea(18, 50); out.setEditable(false);
        JButton b = new JButton("Generate Merit Lists");
        b.addActionListener(e -> new Thread(() -> {                // background thread keeps GUI responsive
            try {
                Map<Program, List<Admission>> m = sys.generateAll((Integer) yr.getSelectedItem());
                StringBuilder sb = new StringBuilder();
                m.forEach((p, l) -> { sb.append(p.dept + " / " + p.name + " (" + p.type() + ")\n");
                    int i = 1; for (Admission a : l) sb.append(String.format("  %d. %s  %.1f%n", i++, a.student.na
me, a.merit)); });
                SwingUtilities.invokeLater(() -> out.setText(sb.toString()));
            } catch (InterruptedException ex) { }
        }).start());
        JPanel top = new JPanel(); top.add(new JLabel("Academic Year:")); top.add(yr); top.add(b);
        f.add(top, BorderLayout.NORTH); f.add(new JScrollPane(out));
        f.pack(); f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); f.setVisible(true);
    }
    public static void main(String[] args) throws Exception {
        if (args.length > 0 && args[0].equals("demo")) {            // console test
            AdmissionSystem sys = sample();
            System.out.println("Admissions 2026: " + sys.byYear(2026).size());
```

```
            sys.generateAll(2026).forEach((p, l) -> { System.out.println(p.dept + " / " + p.name);
                l.forEach(a -> System.out.printf("   %s %.1f%n", a.student.name, a.merit)); });
        } else SwingUtilities.invokeLater(AdmissionSystem::gui);
    }
}
```

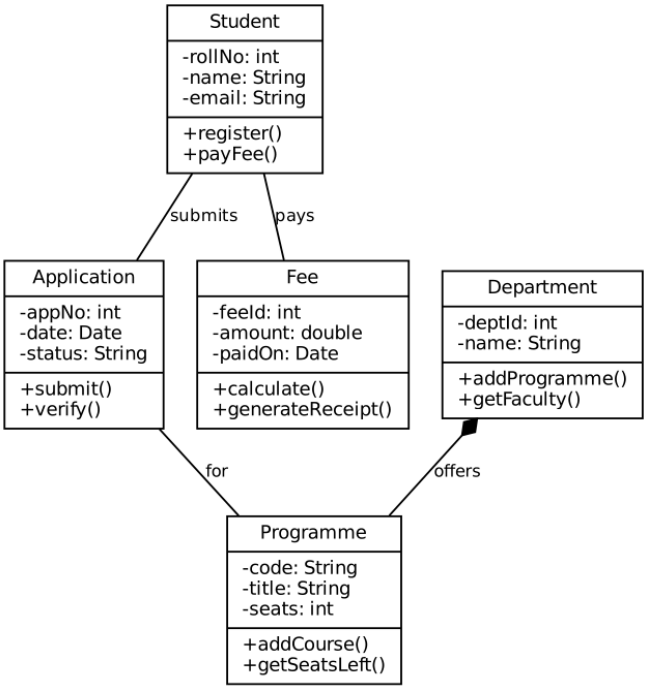
```
Output (java AdmissionSystem demo):
Admissions 2026: 5
SCSIT / M.Tech (IASE)
   Rohit 80.8
   Mahesh 80.0
Management / MBA
   Karan 81.0
SCSIT / B.Tech (CS)
   Neha 80.8
```

Run without arguments to open the Swing window. Console demo was run; the GUI window itself could not be opened here.

Section C: StarUML

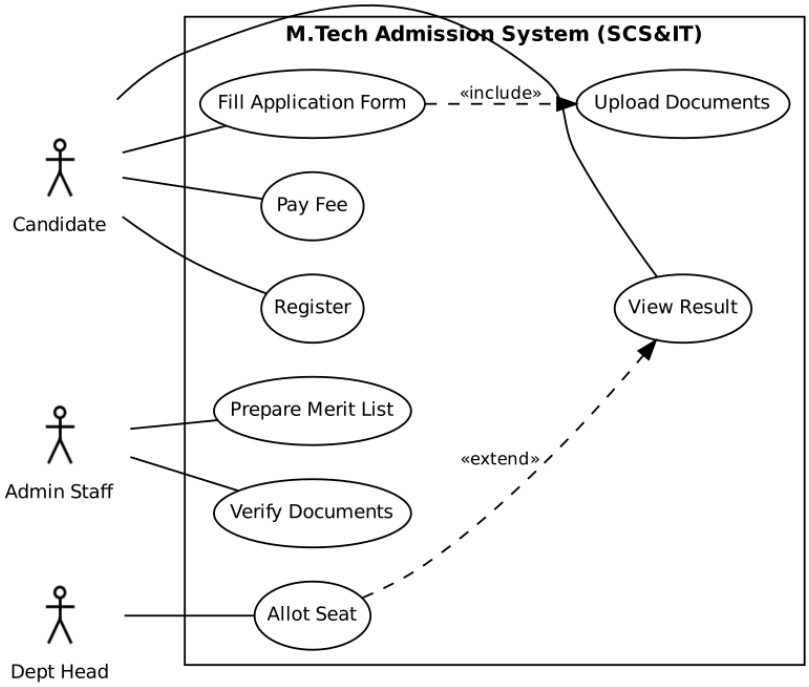
Diagrams are drawn in UML notation (can be redrawn one-to-one in StarUML).

Q1. Five objects with attributes and operations

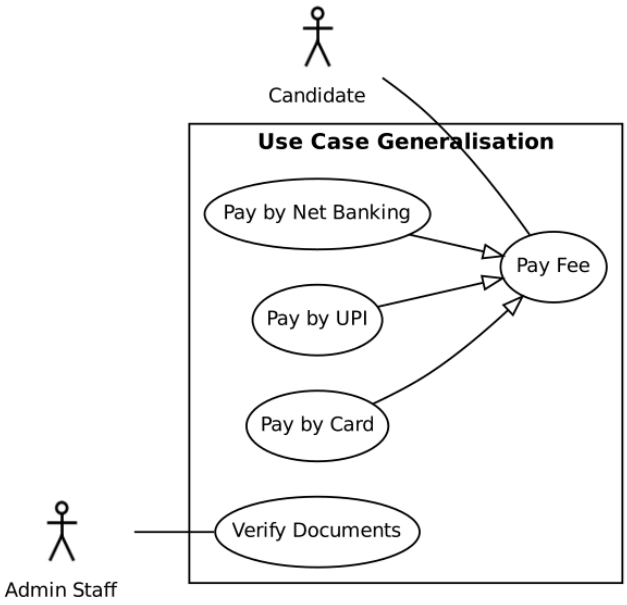


Student, Application, Fee, Programme, Department

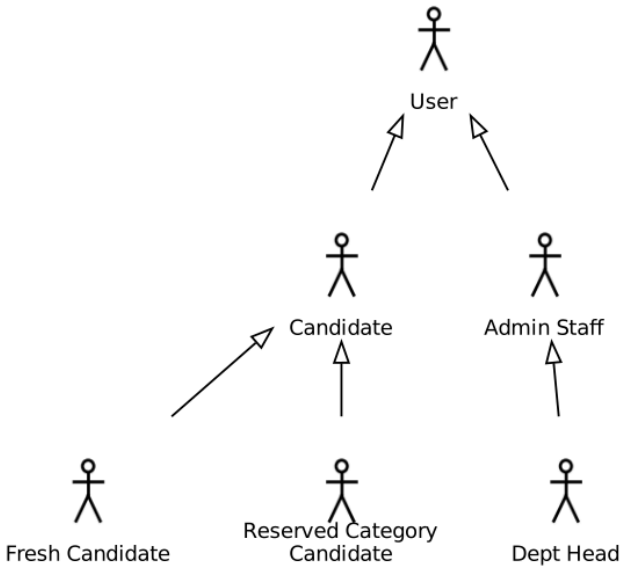
Q2. Use case model for M.Tech admission in SCS&IT



(1) Use case model

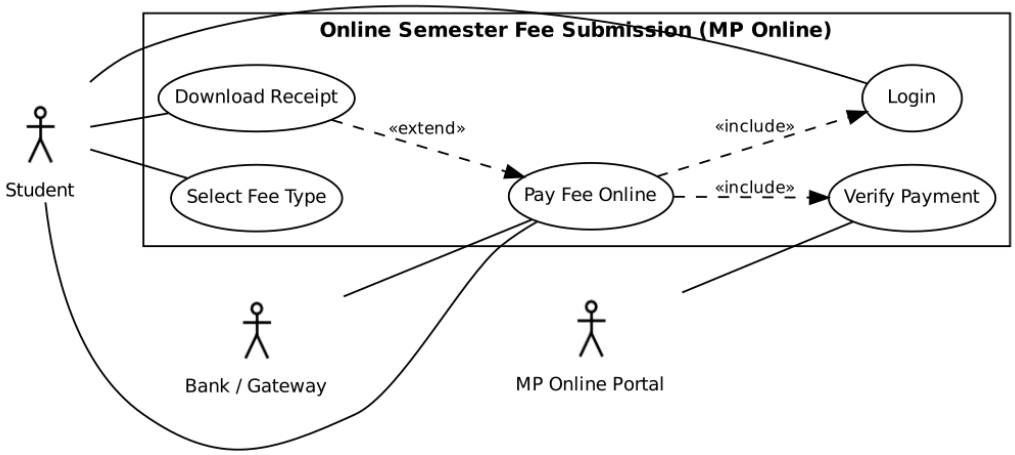


(2) Use case generalisation (Pay Fee)

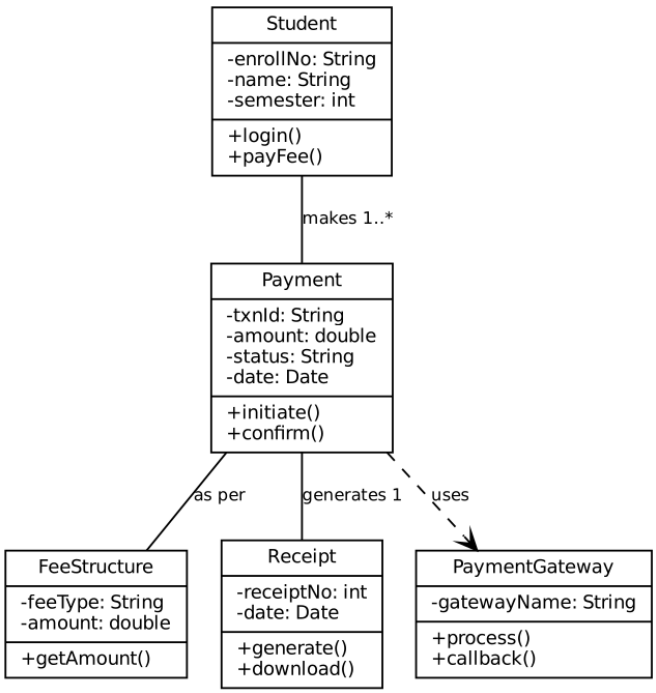


(3) Actor generalisation

Q3. OOSE – online semester fee submission via MP Online

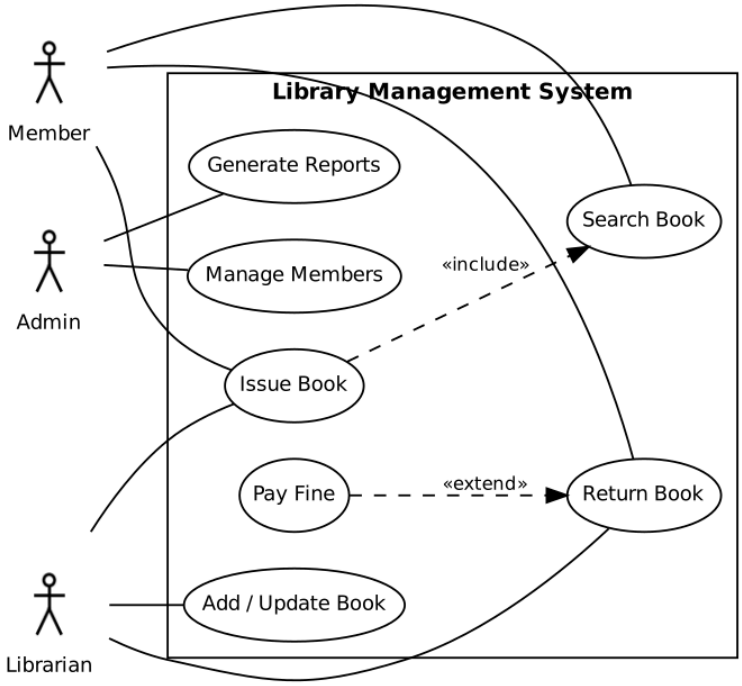


Use case model

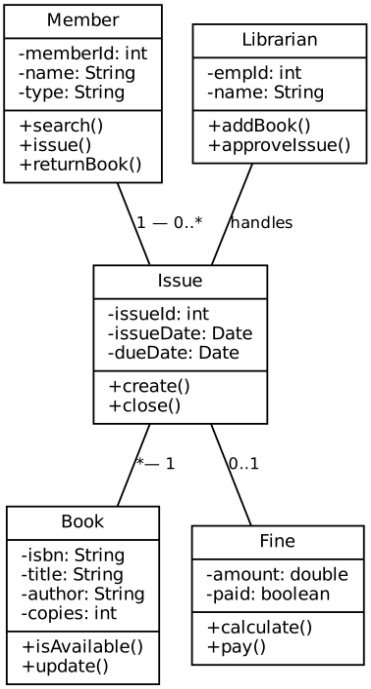


Class diagram

Q4. OOSE – Library Management System

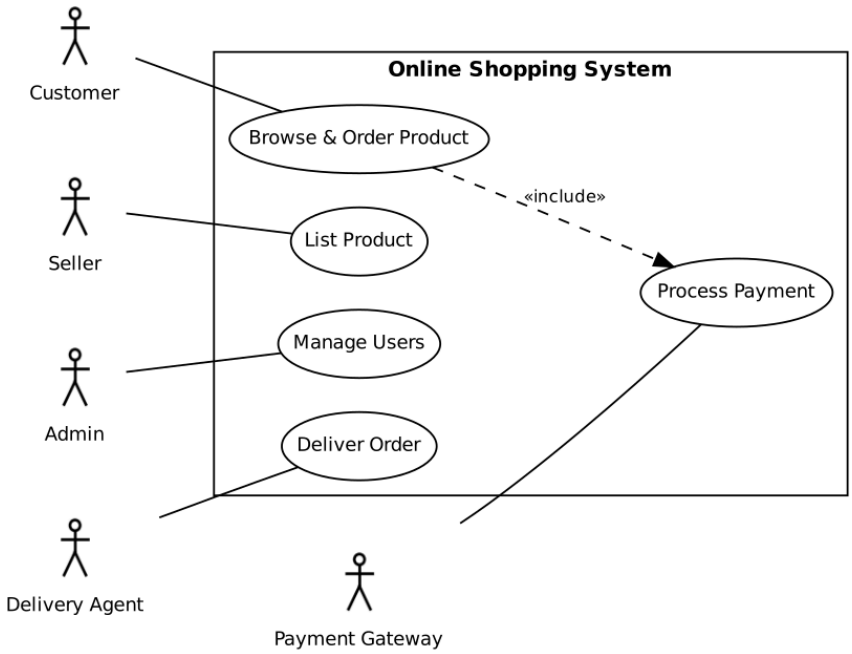


Use case model



Class diagram

Q5. Five actors and five use cases (domain: Online Shopping)



Actors: Customer, Seller, Admin, Delivery Agent, Payment Gateway

Domain was not specified in the question; Online Shopping is used as the example.

Section D: Download / Reading

Q1. Paper: "Software Reuse: Research Roadmap – Present, Past, Future"

An exact match for this title was not found online. Closest papers on the same topic:

- W. B. Frakes and K. Kang, "Software reuse research: status and future", *IEEE Transactions on Software Engineering*, 31(7), 2005. DOI: 10.1109/TSE.2005.85
- "Opportunities for Software Reuse in an Uncertain World: From Past to Emerging Trends", *Journal of Software: Evolution and Process* (DOI 10.1002/smr.2217). Free PDF: https://www.es.mdh.se/pdf_publications/5550.pdf

Download the paper from the link or DOI above and attach it with the assignment.